

LocalAI TV — Admin Dashboard & Broadcast Infrastructure

Final Specification Document · v1.3

Status:  Integration contract LOCKED — AI Pipeline DB design adopted, all engineer questions answered **Last updated:** 15 May 2026 **Owner:** Koneti Mohan Reddy (localaitv@gmail.com) **Replaces:** v1.2 (14 May 2026) **Scale target:** 9 channels (today) → 300 channels (Month 1) → 3,000 channels (Month 6, pan-India) **Daily volume target:** 200 videos/day per constituency = 600,000 videos/day at peak **Document purpose:** Final shareable spec — integration contract locked, ready for Phase 0/1 build.

v1.3 changes (from Gyan's DB spec + Sameer's clarification note, 15 May 2026)

-  **Two-database federation model** (§7c) — Admin Dashboard DB (Supabase) ↔ AI Pipeline DB (AI team's infra), linked by `content_id`
-  **AI Pipeline DB schema** (§7b) — Gyan's 10 first-phase tables adopted verbatim (`ai_processing_jobs` , `content_assets` , `content_collections` , scheduler tables, `ai_callbacks_outbox` , etc.)
-  **DB-driven scheduler** (§13 rewrite) — Gyan's `program_schedule` / `schedule_slots` / `schedule_overrides` replaces v1.2's simpler `publishing_queue`
-  **`content_collections`** logical-folder model (§8) — scheduler calls content by collection, never by physical S3 path
-  **`ai_callbacks_outbox`** transactional outbox pattern (§15) — reliable AI→Admin callback delivery
-  **Staging + production webhook/callback URLs** (§15) — env-configurable, Phase 1 tests on staging
-  **HMAC secret + callback token exchange runbook** (§16) — Supabase Vault + AWS Secrets Manager + 1Password
-  **4xx vs 5xx callback contract** (§15.7) — 5xx retry (transient), 4xx dead-letter (permanent), 30-min watchdog reconciliation
-  **EC2 migration confirmed lift-and-shift** (§20 Phase 0) — zero logic changes, one NVENC flag swap
-  **Sameer's committed delivery plan** (§20) — task ownership between Sameer + Gnana incorporated
- All v1.2 broadcast-scale architecture preserved** (AWS EC2 GPU, S3 + Cloudflare CDN, Hostinger VPS admin, YouTube live-only)

Version History

Version	Date	Changes
v1.0	24 May 2026	Initial plan, 24 sections
v1.1	14 May 2026	Consolidated 3 engineering reviews (Abishek/Sameer/Gnana). Added production reliability, observability, security hardening, integration contract.
v1.2	14 May 2026	Broadcast-scale rebuild: AWS EC2 GPU compute, Cloudflare CDN, S3 lifecycle policies, Hostinger VPS for admin only, YouTube for live channels only, 3,000-channel scale plan with phased rollout (Phase 0/1/2/3).

Key changes from v1.1 → v1.2 (founder-confirmed decisions D1-D8)

-  **Hosting clarified** — Hostinger VPS for admin dashboard (NOT Netlify); AWS EC2 GPU for AI Pipeline (migrating from shared VPS — fixes 15min→90sec bulletin generation bottleneck)
-  **Storage finalized** — AWS S3 Mumbai with automatic lifecycle policies (Standard → IA → Glacier Instant → Glacier Deep Archive)

3.  **CDN mandatory** — Cloudflare CDN in front of S3 (saves ₹5-9 crore/month at peak scale by serving 90%+ from cache)
 4.  **YouTube role clarified** — for LIVE channels only; all other content (Mana Kurnool Shorts, Kurnool Local, news bulletins, Mana Kurnool Prasaru, national/state/district news) served from S3 via Cloudflare CDN
 5.  **Scale projections added** — 3,000 channels × 200 videos/day = 600K videos/day at peak (Month 6)
 6.  **Cost projections rebuilt** — broadcast-scale economics (₹14L/mo at 300 channels, ₹1.3 CR/mo at 3,000 channels)
 7.  **Phased rollout** — Phase 0 (this week: EC2 GPU migration) → Phase 1 (300 channels) → Phase 2 (1,000 channels) → Phase 3 (3,000 channels)
 8.  **GPU compute** — AWS EC2 g4dn.xlarge (NVIDIA T4, Mumbai) for AI Pipeline; Auto Scaling Group based on queue depth
 9. **AWS retained** — team familiarity outweighs marginal GCP savings; no migration to GCP
 10. **All v1.1 enhancements preserved** — queue infrastructure, observability, security hardening, idempotency, status enum, Gnana's bulletin output structure
-

Table of Contents

1. [Executive Summary](#)
 2. [System Architecture Overview](#)
 3. [Three-Tier Admin System](#)
 4. [Content Workflow \(End-to-End\)](#)
 5. [The 8 Upload Forms — Field Audit](#)
 6. [NotebookLM Integration](#)
 7. [Database Architecture](#)
 8. [Storage Folder Structure](#)
 9. [Hidden Admin Access](#)
 10. [API Endpoints](#)
 11. [Admin Dashboard UI](#)
 12. [Analytics Dashboard with Drill-Down](#)
 13. [Channel Publishing Automation & Time Slots](#)
 14. [Push Notification Templates](#)
 15. [AI Pipeline Webhook Integration \(Updated\)](#)
 16. [Security & Row Level Security](#) 16a.  **Security Hardening**
 17. [Audit Log](#) 17a.  **Queue Infrastructure** 17b.  **Observability & Monitoring**
 18. [Tech Stack Summary](#)
 19. [Cost Projections](#)
 20. [Implementation Phases](#) 20a.  **Production Readiness Checklist**
 21. [Team Requirements](#)
 22. [What's IN / OUT of v1](#)
 23. [What I Need From You — Action Items](#)
 24. [Approval & Sign-off](#)
-

1. Executive Summary

We are building **broadcast-scale infrastructure** for LocalAI TV that:

- Provides a 3-tier admin team (super → master → admin) reviewing citizen submissions before AI processing
- Integrates with the **AI Pipeline (FastAPI + Celery + RabbitMQ + Redis)** which migrates from Hostinger VPS to **AWS EC2 GPU** (g4dn.xlarge) for 8-10× faster bulletin generation

- Stores all content in **AWS S3 (Mumbai)** with automatic lifecycle policies (Standard → IA → Glacier)
- Delivers content to mobile apps via **Cloudflare CDN** (free tier covers Phase 1) — mandatory at scale
- Uses **YouTube ONLY for live 24/7 channels** (free CDN + storage for the most bandwidth-heavy content)
- Supports state-restricted admins for multi-language operation across India
- Integrates Google NotebookLM-generated content as a separate auto-approved stream
- Provides time-slot-based publishing automation with push notifications to uploaders

Scale targets:

- **Today:** 9 channels (AP + TG)
- **Month 1:** 300 channels (AP + TG constituency-level)
- **Month 6:** 3,000 channels (pan-India, all states)
- **Daily upload:** 200 videos/channel = up to 600,000 videos/day at peak
- **Storage growth:** ~20 PB after Year 1 at full scale

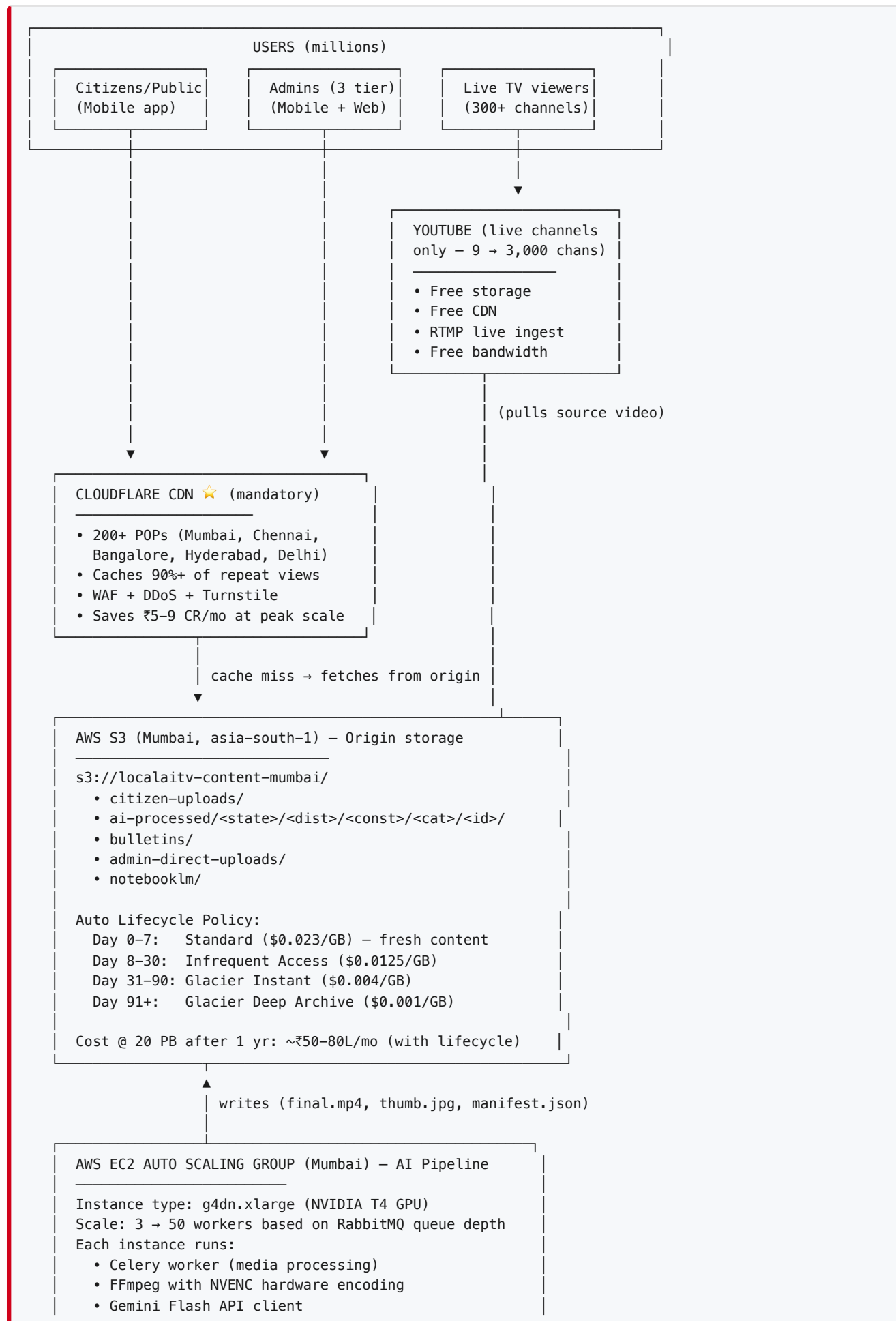
Monthly infrastructure cost:

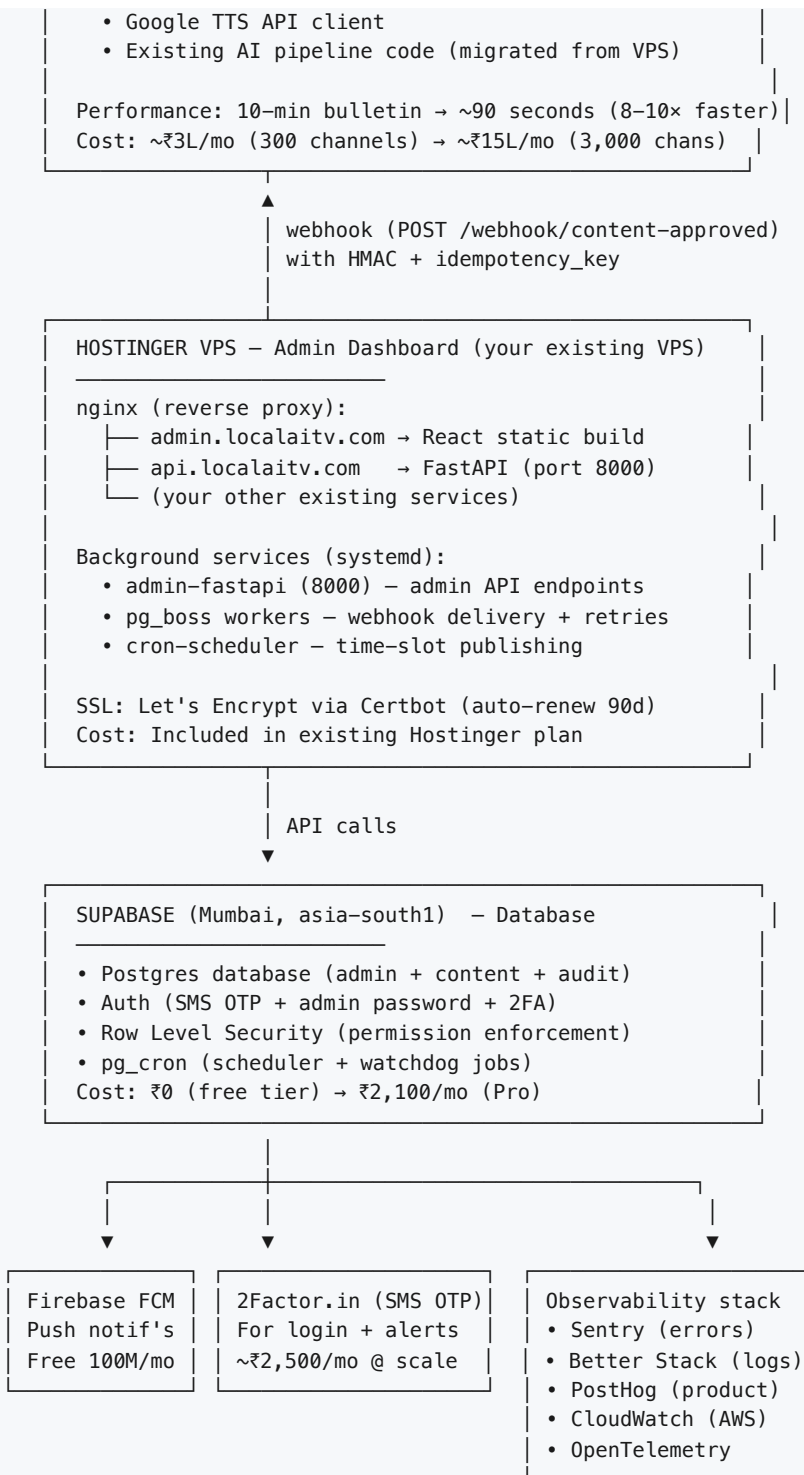
- **Today (9 channels):** ~₹50K
- **Month 1 (300 channels):** ~₹14L
- **Month 6 (3,000 channels):** ~₹1.3 CR (with Cloudflare CDN — otherwise ₹5-10 CR without)

Team requirements:

- Founder (decisions, API keys, approval)
 - AI Team (existing — migrates code from Hostinger VPS to AWS EC2)
 - Claude (Anthropic) — admin dashboard build
 - DevOps engineer: NOT required for v1 (managed services + AWS Auto Scaling Groups handle ops)
-

2. System Architecture Overview (v1.2 — Broadcast Scale)





Architecture summary (per founder decisions D1-D8)

Component	Decision	Where it runs
AI Pipeline compute	D1 ✓	AWS EC2 g4dn.xlarge (Mumbai) — migrates from Hostinger VPS
Storage	D2 ✓	AWS S3 (Mumbai) with auto lifecycle policies
CDN	D2 ✓	Cloudflare in front of S3 — mandatory for cost
Cloud provider	D3 ✓	AWS (not GCP) — team familiarity
YouTube	D4 ✓	LIVE channels only — all other content via CDN
Admin Dashboard hosting	D5 ✓	Hostinger VPS (your existing)
S3 lifecycle policies	D6 ✓	From Day 1 (Standard → IA → Glacier)
Rollout	D7 ✓	Phased: Phase 0 (this week) → 1 → 2 → 3

3. Three-Tier Admin System

3.1 Roles

Role	Description
Super Admin	Founder-level. Full access. Only 1 person.
Master Admin	Senior moderator. Can create admins. Full state access.
Admin	Front-line moderator. Restricted to specific assigned state(s).
Citizen / Public	Unified role. No distinction between "citizen reporter" and "public".

3.2 Permissions Matrix

Capability	Super	Master	Admin
Create Master Admin	✓	✗	✗
Create Admin	✓	✓	✗
Suspend admins below tier	✓	✓ (admins only)	✗
Geographic access	All	All	Assigned states only
Upload (national/state/district/constituency)	✓	✓	✓ (within assigned states)
Bypass review (own uploads auto-approved)	✓	✓	✓
NotebookLM upload (5 sub-types)	✓	✓	✓
Approve / Modify+Approve / Reject	✓	✓	✓
Escalate to higher tier	n/a (top)	→ Super	→ Master
View all-India analytics	✓	✓	✗
Cancel approved jobs	✓	✓	✗
Audit log access	All	Team only	Own only

3.3 State Restriction (Multi-Language Support)

Admins are assigned one or more states. Queue auto-filters; RLS enforces at DB level.

Admin: Ravi Kumar

└ Phone: +91 98765 43210

└ Language access (multi-select):

- ☒ Andhra Pradesh (Telugu)
- ☒ Telangana (Telugu)
- ☐ Karnataka (Kannada)
- ☐ Tamil Nadu (Tamil)

4. Content Workflow (End-to-End)

STEP 1 – UPLOAD

Citizen submits: 1–3 media files + headline + description + location

- Files → Cloudflare R2 (Cloudflare Turnstile blocks bots)
- ClamAV virus scan via Edge Function
- SHA-256 hash dedup
- NSFW check via Google Cloud Vision (flag for priority review)
- Metadata → Supabase (status: pending)



STEP 2 – ADMIN CLAIM (atomic lock)

Admin clicks item:

```
UPDATE content SET claimed_by=$me, claimed_at=NOW()  
WHERE id=$id AND status='pending'  
AND (claimed_by IS NULL OR claimed_at < NOW() - INTERVAL '30 min')  
RETURNING *;
```

- If 1 row returned: this admin owns it
- If 0 rows: another admin already claimed → "already claimed by X"



STEP 3 – DECISION

- ✅ APPROVE
- ✏️ MODIFY + APPROVE (original preserved read-only)
- ❌ REJECT (with reason; citizen notified)
- ESCALATE (admin → master → super)

(only if approved)



STEP 4 – WEBHOOK DELIVERY VIA QUEUE

Admin enqueues webhook job in pg_boss
Worker picks up → POST to AI Pipeline with HMAC + idempotency_key
Retry policy: 1s → 5s → 30s → 5min → 30min
Dead-letter after 5 attempts → Sentry alert + super admin notify



STEP 5 – AI PIPELINE (existing FastAPI + Celery + RabbitMQ + Redis)

FastAPI validates payload + HMAC
Returns 202 Accepted within 200ms
Enqueues Celery task with idempotency_key as task_id
Worker: Gemini Flash → Google TTS → FFmpeg
Output stored: ai-processed/<state>/<dist>/<const>/<cat>/<id>/



STEP 6 – CALLBACK TO ADMIN

POST /api/webhook/ai-processed
Body: { content_id, ai_output_url, thumbnail_url, status, metrics }
Admin marks content: status = 'ready_for_bulletin'
Retry 5 times if Admin unreachable; held in AI's local DLQ otherwise



STEP 7 – TIME-SLOT SCHEDULING


```
pg_cron + pg_try_advisory_xact_lock(slot_id)
Catch-up job for missed slots
Birthday wishes auto-scheduled for birthday date
Other content distributed across 4-5 daily slots
10 min before slot: push notification to uploader
```



```
STEP 8 – BROADCAST
Bulletin assembled (your existing 10-15min compilation)
Stored: ai-processed-bulletins/<state>/<dist>/<const>/bul_<TS>/
Live TV broadcast
Audit log records aired_at
```

5. The 8 Upload Forms — Field Audit

Note: Final field-by-field audit will be done during Phase 1 against the canonical app source ([286.App_v3.jsx](#)). Megan (form engineer) has independently audited the live forms and will align them per founder's decisions. The JSONB `form_data` field handles any form-specific fields without requiring schema changes.

Common rules across ALL forms:

- **Media:** Maximum 3 files (photos/videos in any combination)
 - Exception: Cars max 5, Rentals max 10 (subject to founder decision)
- **Single-language fields:** Headline / description are ONE field each (Telugu OR English; auto-detected)
- **Voice input** supported
- **Required confirmation:** "I confirm content is original" checkbox

A. 📰 News

- Media (1–3, required)
- Headline (single, required, 4-30 words)
- Description (single, required, 20-300 words)
- Location (auto-filled with constituency)

B. 🎂 Birthdays

- Recipient: name, DOB, age (auto), photos (max 3, first required)
- Up to 3 wishers: name + relation (one required), photo (optional)
- Auto-scheduled for birthday date

C. 🏠 Marriage Day (Anniversary)

- Couple name (combined field, required)
- Anniversary date
- Years auto-calculated
- Couple photos (first required)
- Up to 3 senders
- Greeting message (optional — to be ADDED to form per docs)

D. 💍 Upcoming Marriage

- Bride name, Groom name (required)

- Marriage date, Venue, Muhurtham/Reception time
- Invitation card image, Invitation video
- Family contact phone (required)

E. Events

- Event name, Event type (dropdown), Date, Time, Venue
- Description (required), Organizer name (required)
- Contact phone (required)
- Photos (max 3)

F. Shopping

- Shop name OR Item name (one required — founder decision pending)
- Category (dropdown), Price (required), Description
- Advertisement mode (Text / Voice)
- Media (Image / Video / Audio, max 3)
- Seller phone, Location

G. Cars / Vehicles

- Vehicle type, Brand, Model, Year
- KM driven, Fuel type (including Hybrid), Price, Owner type
- Color, Description, Seller details, Address
- Photos (max 5 — special exception)

H. Rentals

- Property type, Configuration (1BHK/2BHK/etc), Furnished status
- Monthly rent, Security deposit (optional)
- Area, Description, Owner name + contact
- Photos (max 10 — special exception)

I. Jobs

- Job title, Company name, Job type (FT/PT/Contract/Daily wage/Internship/Fresher)
- Monthly salary (optional), Location
- Qualification (required), Skills (optional)
- Job description (required), No. of posts
- Contact phone OR email (at least one required)
- Last date to apply (optional)

Future-proofing

When adding new categories (e.g. "Who's Who"): just add to category enum + define fields. Database `form_data` JSONB handles them automatically.

6a. YouTube Live Channel Integration

Role: YouTube is used **ONLY for live 24/7 channels**. All other content (Mana Kurnool Shorts, Kurnool Local, news bulletins, Mana Kurnool Prasarakalu, national/state/district news) is served from AWS S3 via Cloudflare CDN directly to the mobile app.

Why YouTube for live only

Aspect	YouTube (live)	S3 + Cloudflare (other content)
Bandwidth cost	Free (YouTube absorbs)	₹0 with 90%+ CDN cache hit
Storage cost	Free (YouTube absorbs)	We pay ~₹0.50/GB after lifecycle tiering
Quality control	YouTube algorithm decides	Full control
Analytics	YouTube Studio	PostHog + Cloudflare Analytics
Content survives if YT bans channel	Risk: lost	Always in S3
Geographic restrictions	YouTube's CDN rules	Our control via Cloudflare

Operational reality: YouTube is the cheapest way to do 24/7 live distribution at scale, but you should NOT depend on it for short-form content where you want control.

Architecture

9 → 300 → 3,000 LIVE CHANNELS

```
AWS S3 (Mumbai) – Source video
↓
RTMP streaming server on Hostinger VPS (cron-based)
↓
YouTube Live (RTMP ingest endpoint per channel)
↓
YouTube CDN (free, global)
↓
Mobile app embeds YouTube live URL via iframe
```

Database additions

```
ALTER TABLE content ADD COLUMN youtube_video_id TEXT;
ALTER TABLE content ADD COLUMN youtube_watch_url TEXT;
ALTER TABLE content ADD COLUMN youtube_uploaded_at TIMESTAMPTZ;

CREATE TABLE youtube_channels (
  id SERIAL PRIMARY KEY,
  channel_name TEXT NOT NULL,           -- e.g. "Kurnool TV"
  state TEXT NOT NULL,
  district TEXT,
  constituency TEXT,                   -- NULL = district-level
  yt_channel_id TEXT UNIQUE,           -- YouTube's channel ID
  yt_stream_key TEXT,                  -- encrypted in Supabase Vault
  yt_credentials_ref TEXT,             -- OAuth token reference
  is_active BOOLEAN DEFAULT true,
  current_viewer_count INT DEFAULT 0,
  last_stream_started_at TIMESTAMPTZ
);
```

YouTube Data API integration

YouTube Data API v3 quotas (default 10,000 units/day per project):

- Video upload: 1,600 units per upload
- = ~6 uploads per day per project at default quota
- **Need to request quota increase** for 200/day per channel × 3,000 channels = 600,000 uploads/day
- Or distribute across multiple Google Cloud projects

Live streaming approach

For 24/7 live channels, three options:

Option A: Scheduled playlist (simplest, recommended)

- Each YouTube channel runs as a "scheduled live stream"
- RTMP encoder on Hostinger VPS / AWS EC2 streams from S3 source
- Auto-loops through latest bulletins
- Free, no separate streaming server

Option B: Premier-as-Live

- Upload pre-rendered bulletin to YouTube
- Schedule as "Premiere" → appears as live at specified time
- Each premiere = single video
- Limited (300+ premieres/day across all channels)

Option C: Dedicated streaming server

- Self-host with Nginx-RTMP or AWS MediaLive
- Most control but most expensive
- Defer to Phase 3

Recommendation: Option A for v1.

6. NotebookLM Integration

5 content streams from Google NotebookLM — admin uploads, auto-approved, bypasses AI pipeline:

Source type	Description	Duration	Geographic level
Individual news	Constituency-specific items	30 sec – 1 min	Constituency
District bulletin	4-5 news compiled	4-5 min	District
State bulletin	State-wide bulletin	4-5 min	State
National bulletin	National bulletin	4-5 min	National
Debate (any level)	Discussion format	Variable	All 4 levels

Upload: Only Super/Master/Admin (admin within their states only). Auto-approved. Direct-publish lane. `processing_mode = 'notebooklm_generated'`.

7. Database Architecture

Tech: Supabase (managed PostgreSQL) in Mumbai region.

7.1 Updated Tables

`users`

```

CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone       TEXT UNIQUE NOT NULL,
  name        TEXT,
  email       TEXT,
  role        user_role NOT NULL,
  assigned_states TEXT[],
  password_hash TEXT,
  profile_photo_url TEXT,
  constituency TEXT,
  is_verified BOOLEAN DEFAULT false,
  created_by  UUID REFERENCES users(id),
  suspended   BOOLEAN DEFAULT false,
  -- Security hardening (Abishek's input)
  failed_login_count INT DEFAULT 0,
  locked_until TIMESTAMPTZ,
  last_login_at TIMESTAMPTZ,
  last_login_ip INET,
  created_at   TIMESTAMPTZ DEFAULT NOW()
);

CREATE TYPE user_role AS ENUM (
  'super_admin', 'master_admin', 'admin', 'citizen'
);

```

content (updated with v1.1 columns)

```

CREATE TABLE content (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id         UUID REFERENCES users(id),
  author_role       user_role NOT NULL,
  content_source     content_source NOT NULL,
  processing_mode    processing_mode_enum NOT NULL, -- NEW from Gnana
  geography_level    geography_level NOT NULL,
  state             TEXT,
  district          TEXT,
  constituency       TEXT,
  category           content_category NOT NULL,
  target_channel     TEXT, -- NEW for TTS routing
  form_data          JSONB NOT NULL,
  media_urls         JSONB,
  media_hashes       TEXT[], -- NEW SHA-256 for dedup
  nsfw_score         FLOAT, -- NEW from Google Vision
  original_data      JSONB,
  status             content_status NOT NULL DEFAULT 'pending',
  approved_version   INT DEFAULT 1, -- NEW from Gnana
  idempotency_key    TEXT UNIQUE, -- NEW from Gnana
  claimed_by         UUID REFERENCES users(id),
  claimed_at         TIMESTAMPTZ,
  reviewed_by        UUID REFERENCES users(id),
  reviewed_at        TIMESTAMPTZ,
  rejection_reason   TEXT,
  escalated_to       UUID REFERENCES users(id),
  webhook_sent_at    TIMESTAMPTZ, -- NEW prevent double-send
  ai_job_id          TEXT, -- NEW Celery task id
  ai_callback_at     TIMESTAMPTZ, -- NEW
  ai_output_url      TEXT, -- NEW
  ai_thumbnail_url   TEXT, -- NEW
  ai_duration_seconds INT,
  ai_metrics         JSONB, -- NEW timing metrics
  created_at         TIMESTAMPTZ DEFAULT NOW(),
  updated_at         TIMESTAMPTZ DEFAULT NOW(),
  deleted_at         TIMESTAMPTZ
);

CREATE TYPE content_source AS ENUM (
  'citizen', 'admin_direct',
  'notebooklm_news', 'notebooklm_bulletin', 'notebooklm_debate'
);

CREATE TYPE processing_mode_enum AS ENUM (
  'ai_pipeline', -- needs Gemini/TTS/FFmpeg
  'raw_video', -- admin direct (skip AI)
  'notebooklm_generated' -- NotebookLM (skip AI)
);

CREATE TYPE geography_level AS ENUM ('national', 'state', 'district', 'constituency');

CREATE TYPE content_category AS ENUM (
  'news', 'birthday', 'marriage_day', 'upcoming_marriage',
  'event', 'shopping', 'car_sale', 'rental', 'job'
);

-- Updated status enum per Gnana's recommendation
CREATE TYPE content_status AS ENUM (
  'pending', 'in_review', 'rejected', 'escalated',
  'approved', 'queued', 'processing',
  'failed_retryable', 'failed_final',
  'ready_for_bulletin', 'scheduled', 'published',
  'cancelled', 'callback_lost', 'archived'
);

```

 **webhook_deliveries** (queue + retry tracking)

```
CREATE TABLE webhook_deliveries (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content_id        UUID REFERENCES content(id),
  idempotency_key   TEXT UNIQUE NOT NULL,
  endpoint_url      TEXT NOT NULL,
  payload           JSONB NOT NULL,
  attempt_count     INT DEFAULT 0,
  last_attempted_at TIMESTAMPTZ,
  last_response_status INT,
  last_response_body TEXT,
  delivered_at      TIMESTAMPTZ,
  dead_lettered_at  TIMESTAMPTZ,
  created_at        TIMESTAMPTZ DEFAULT NOW()
);
```

NEW **ai_callbacks** (inbound from AI Pipeline)

```
CREATE TABLE ai_callbacks (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content_id        UUID REFERENCES content(id),
  job_id           TEXT,
  idempotency_key   TEXT,
  status           TEXT,
  ai_output_url     TEXT,
  thumbnail_url     TEXT,
  duration_seconds  INT,
  error_message     TEXT,
  metrics          JSONB,
  received_at      TIMESTAMPTZ DEFAULT NOW()
);
```

content_audit_log (unchanged from v1.0)

[Same as v1.0 — see §7 of original plan]

publishing_queue (unchanged from v1.0)

[Same as v1.0]

notifications (unchanged)

[Same as v1.0]

7.2 Indexes (updated)

```
-- Fast admin queue queries (state-filtered)
CREATE INDEX idx_content_queue ON content(status, state, created_at)
  WHERE status IN ('pending', 'in_review') AND deleted_at IS NULL;

CREATE INDEX idx_content_geo ON content(state, district, constituency);

CREATE INDEX idx_approved_pool ON content(constituency, category, created_at DESC)
  WHERE status IN ('approved', 'ready_for_bulletin');

CREATE INDEX idx_reporter_stats ON content(author_id, status, created_at);

CREATE INDEX idx_publishing_pending ON publishing_queue(scheduled_for_date, scheduled_for_slot)
  WHERE status = 'queued';

-- For media dedup
CREATE INDEX idx_media_hashes ON content USING GIN (media_hashes);

-- For form-specific JSONB search
CREATE INDEX idx_form_data_gin ON content USING GIN (form_data);

-- For webhook delivery monitoring
CREATE INDEX idx_webhook_pending ON webhook_deliveries(content_id, attempt_count)
  WHERE delivered_at IS NULL AND dead_lettered_at IS NULL;
```

7.3 Row Level Security (unchanged from v1.0 — see §16)

7b. AI Pipeline Database (Gyan's design — adopted verbatim)

The AI Pipeline has its **own database** (separate from the Admin Dashboard DB). This is the AI team's existing infrastructure, extended with the following 10 first-phase tables per Gyan's specification.

7b.1 **ai_processing_jobs** — job lifecycle tracking

```
id                -- primary key
content_id        -- FK link to Admin Dashboard content (the federation key)
approved_version
idempotency_key   -- content_id + "_v" + approved_version
processing_mode    -- ai_pipeline | raw_video | notebooklm_direct
status            -- queued|processing|failed_retryable|failed_final|ready_for_bulletin|published
queue_name
worker_role       -- media | build | maintenance
celery_task_id
attempt_count
last_stage        -- gemini | tts | ffmpeg | upload
last_error
accepted_at
started_at
finished_at
last_heartbeat_at
created_at
updated_at
```


7b.2 `content_assets` — every S3 file (input/intermediate/output)

```
id
content_id
ai_job_id
asset_role          -- input_image | input_video | input_audio | generated_tts |
                   -- generated_thumbnail | final_video | final_bulletin |
                   -- notebooklm_direct_output
storage_backend     -- s3
bucket
object_key          -- ai-processed/<state>/<dist>/<const>/<cat>/<content_id>/<asset_role>.<ext>
public_url          -- via Cloudflare CDN
mime_type
size_bytes
checksum            -- SHA-256
lifecycle_status    -- active | archived | expired | deleted
created_at
```

7b.3 `content_collections` — logical folders (decouples scheduler from S3 paths)

```
id
collection_type      -- local_news_bulletin | notebooklm_district_news | classified |
                   -- birthday | marriage | shopping | car_sale | debate |
                   -- local_interview | live_event | school_function | vegetable_rates
state | district | constituency
channel_id
is_active
```

7b.4 Scheduler tables (replaces v1.2 `publishing_queue`)

```
program_schedule:   id | channel_id | schedule_name | active
schedule_slots:    id | schedule_id | start_time | end_time | slot_type |
                   collection_id | filler_policy
schedule_overrides: id | channel_id | slot_id | override_by | override_priority |
                   starts_at | ends_at | selected_asset_id
channel_program_rules: id | channel_id | collection_id | priority | rotation_policy
```

7b.5 Filter tables (3,000-channel scale — precomputed, no live queries)

```
channel_config:      id | channel_id | state | district | constituency |
                   filter_rules (JSONB) | is_active
content_filter_counts: channel_id | collection_type | status |
                   count | last_computed_at (refreshed by cron, NOT live)
```

7b.6 Display rule tables (classified/birthday/marriage auto-expiry & rotation)

```
content_templates:   id | collection_type | template_html | active
content_display_rules: content_id | display_start_at | display_end_at |
                   expiry_action | priority | repeat_allowed | max_plays
content_rotation_state: content_id | channel_id | last_played_at | play_count
```

7b.7 `ai_callbacks_outbox` — transactional outbox (reliable callback delivery)

```
id
ai_job_id
content_id
callback_url
payload
status          -- pending | delivered | dead_lettered
attempt_count
last_response_status
next_retry_at
delivered_at
dead_lettered_at
```

7b.8 Extensions to existing AI Pipeline tables (additive only)

```
news_items:      + content_id, ai_job_id, channel_id, collection_id,
                  + source_type, asset_id, display_start_at, display_end_at
processed_reports: + content_id, ai_job_id, approved_version, last_heartbeat_at
item_events:      + ai_job_id, content_id, channel_id
bulletin_events:  + channel_id, schedule_slot_id, bulletin_asset_id
```

7b.9 First-phase build list (Gyan's discipline — build ONLY these first)

New tables	Existing tables to extend
ai_processing_jobs	processed_reports
content_assets	news_items
content_collections	app_state
program_schedule	item_events
schedule_slots	bulletin_events
schedule_overrides	incidents
channel_config	
content_filter_counts	
content_display_rules	
ai_callbacks_outbox	

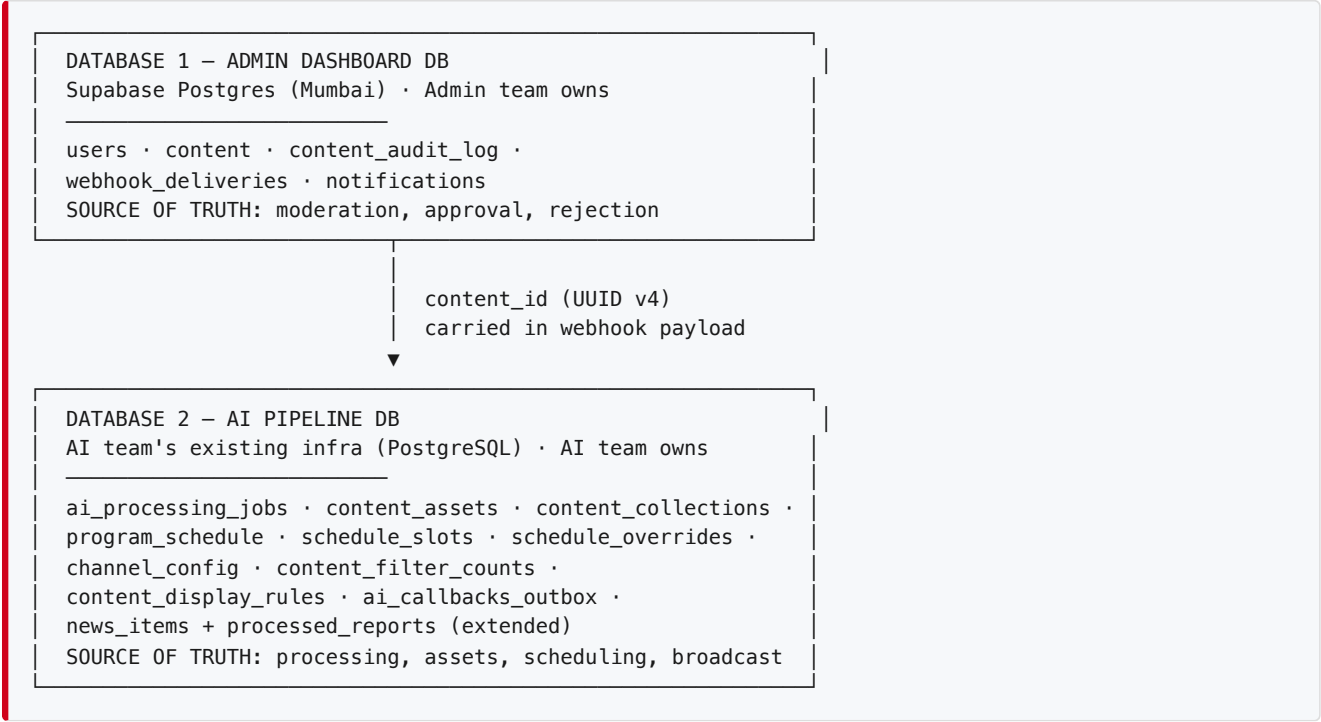
Do NOT build the full set at once. Build these 10, extend the 6, ship Phase 1.

7b.10 Worker / DB interaction model

Worker	DB responsibility
media-worker	Creates/updates <code>ai_processing_jobs</code> ; creates <code>content_assets</code> for inputs + intermediate output; writes <code>news_items</code> when content is bulletin-ready
build-worker	Reads <code>news_items</code> , <code>content_collections</code> , <code>schedule_slots</code> ; creates <code>final_video</code> / <code>final_bulletin</code> assets; writes <code>bulletin_events</code>
maintenance-worker	Checks stale <code>ai_processing_jobs</code> ; checks failed callbacks in <code>ai_callbacks_outbox</code> ; requeues safe retries
celery-beat	Triggers planner, retry, watchdog jobs

7c. NEW Two-Database Federation Model

LocalAI TV has **two databases**, linked by `content_id` :



Why two databases (not one)

Reason	Detail
Separation of concerns	Admin team owns moderation DB; AI team owns processing DB
Different scale profiles	Admin DB = low volume (decisions). AI DB = very high (600K jobs/day)
Failure isolation	If Admin DB briefly down, AI pipeline keeps processing queued work
Team autonomy	Each team evolves its schema independently
Pre-existing	AI team already has <code>processed_reports</code> , <code>news_items</code> , etc. — extend, don't replace

The federation link

`content_id` (UUID v4, generated by Admin Dashboard) is the ONLY shared key:

- Admin generates `content_id` at upload
- On approval, webhook carries `content_id` → AI Pipeline
- AI Pipeline stores it in `ai_processing_jobs.content_id`
- All AI-side tables reference `content_id`
- Callback returns `content_id` → Admin updates `content.status`

Three complementary tracking tables (NOT duplicates)

Table	DB	Tracks
<code>webhook_deliveries</code>	Admin	Did Admin→AI webhook POST succeed?
<code>ai_processing_jobs</code>	AI	Job lifecycle inside pipeline
<code>ai_callbacks_outbox</code>	AI	Did AI→Admin callback POST succeed?

End-to-end traceability across both databases.

Open coordination item (Phase 1 decision)

How does Admin Dashboard write to the **schedule tables** (which live in the AI Pipeline DB)?

- **Option A (recommended):** Admin calls AI-side API `POST https://pipeline.localaitv.com/api/schedule/...`
- **Option B:** Admin writes directly to AI Pipeline DB for those 3 tables only

Recommendation: **Option A** (preserves DB separation). Final decision in Phase 1 with AI team.

8. Storage Folder Structure (v1.3 — AWS S3 + DB-managed collections)

v1.3 change (per Gyan §2.4): Production content is **called by logical collection from the DB, never by physical S3 path**. The S3 folder structure below is the physical layout; the scheduler/build-worker resolves content via `content_collections` + `content_assets` tables (§7b.2-7b.3), not by walking folders. This decouples scheduling from storage and is mandatory for 3,000-channel scale.

AWS S3 bucket: `localaitv-content-mumbai`

```
s3://localaitv-content-mumbai/
├── citizen-uploads/
│   ├── pending/           ← awaiting admin review
│   └── approved/         ← admin-approved, queued for AI
├── admin-direct-uploads/ ← processing_mode='raw_video'
├── notebooklm/
│   ├── individual-news/<state>/<district>/<constituency>/
│   ├── bulletins/
│   │   ├── district/<state>/<district>/
│   │   ├── state/<state>/
│   │   └── national/
│   └── debates/<level>/<state>/<district>/<constituency>/
├── ai-processed/          ← individual AI-processed items
│   ├── <state>/<district>/<constituency>/<category>/<content_id>/
│   │   ├── final.mp4
│   │   ├── headline.mp3
│   │   ├── thumb.jpg
│   │   └── meta.json
├── ai-processed-bulletins/ ← per Gnana's exact spec
│   ├── <state>/<district>/<constituency>/bul_<YYYYMMDD_HHMMSS>/
│   │   ├── final_bulletin.mp4
│   │   ├── bulletin_manifest.json
│   │   ├── thumbnail.jpg
│   │   ├── segments/
│   │   │   ├── item_001.mp4
│   │   │   └── item_NNN.mp4
│   └── metadata.json
```

S3 Lifecycle Policy (mandatory — automatic tiering for cost optimization)

```
# AWS S3 Lifecycle Configuration
# Saves 50-75% on storage costs at scale automatically
rules:
  - id: tier-fresh-content
    prefix: ai-processed/
    transitions:
      - days: 8
        storage_class: STANDARD_IA           # $0.0125/GB (50% cheaper)
      - days: 31
        storage_class: GLACIER_IR            # $0.004/GB (75% cheaper)
      - days: 91
        storage_class: DEEP_ARCHIVE          # $0.001/GB (95% cheaper)

  - id: tier-bulletins
    prefix: ai-processed-bulletins/
    transitions:
      - days: 30
        storage_class: GLACIER_IR
      - days: 180
        storage_class: DEEP_ARCHIVE

  - id: expire-temp
    prefix: temp/
    expiration:
      days: 1

  - id: expire-bulletin-segments
    prefix: ai-processed-bulletins/*/segments/
    expiration:
      days: 7

  - id: keep-manifests
    prefix: ai-processed-bulletins/*/metadata.json
    transitions:
      - days: 30
        storage_class: GLACIER_IR
```

Cloudflare CDN (mandatory in front of S3)

CDN must be deployed before scaling beyond 100 channels. Without it, S3 egress fees become catastrophic at peak scale (₹5-10 crore/month).

```
DNS Configuration:
content.localaitv.com → CNAME → Cloudflare
                        | (cache layer)
                        ▼ (cache miss only)
                        AWS S3 origin
```

Cache rules:

- Photos / thumbnails: Cache 7 days
- AI-processed videos: Cache 30 days (immutable, content-addressed)
- Bulletins: Cache 24 hours
- Audio (MP3): Cache 30 days

Expected cache hit ratio: 90%+ after initial warmup. Means only 10% of requests reach S3 = 10% of egress costs.

9. Hidden Admin Access

- **Web:** <https://admin.localaitv.com> (subdomain, not linked publicly)

- **Mobile:** Hidden 5-tap on Profile reveals "Staff Sign In" OR memorize URL
 - **Login:** Phone/email + password + SMS OTP 2FA via 2Factor.in
 - **Session:** 24h expiry
 - **Failed logins:** Account locked after 5 failures, IP-blocked after 20/hour
-

10. API Endpoints (updated with v1.1 additions)

Auth (admin)

- `POST /api/admin/login` — phone+password → OTP challenge
- `POST /api/admin/verify-otp` — confirm → session
- `POST /api/admin/logout`
- `GET /api/admin/me`
- `POST /api/admin/change-password`

Content queue

- `GET /api/admin/queue?state=&category=&geo_level=`
- `GET /api/admin/content/:id`
- `POST /api/admin/claim/:id`
- `POST /api/admin/release/:id`
- `POST /api/admin/approve/:id`
- `POST /api/admin/modify-approve/:id`
- `POST /api/admin/reject/:id`
- `POST /api/admin/escalate/:id`

Admin uploads

- `POST /api/admin/upload` (auto-approved, AI pipeline)
- `POST /api/admin/upload-notebooklm` (auto-approved, direct publish)

Citizen

- `POST /api/citizen/upload`
- `GET /api/citizen/my-uploads`
- `GET /api/citizen/notifications`

User management

- `GET /api/admin/users?role=`
- `POST /api/admin/users`
- `PATCH /api/admin/users/:id`
- `DELETE /api/admin/users/:id`

Analytics

- `GET /api/admin/analytics/dashboard?from=&to=&state=&category=`
- `GET /api/admin/analytics/breakdown?dimension=`
- `GET /api/admin/analytics/reporter/:id`
- `GET /api/admin/analytics/items?...`

Publishing

- `GET /api/admin/publishing/queue`

- `POST /api/admin/publishing/schedule`
- `POST /api/admin/publishing/cancel/:id`

Webhook + Cancel job

- `POST /webhook/content-approved` (Admin → AI Pipeline)
- `POST /api/webhook/ai-processed` (AI Pipeline → Admin)
- `DELETE /webhook/cancel-job/:job_id` (Admin → AI Pipeline) 
- `GET /api/admin/job-status/:job_id` (Admin polls AI) 

Audit

- `GET /api/admin/audit?content_id=`
- `GET /api/admin/audit/recent`

11. Admin Dashboard UI

(Unchanged from v1.0 — see §11 of original plan)

12. Analytics Dashboard with Drill-Down

(Unchanged from v1.0 — see §12)

13. Channel Publishing Automation & Time Slots

Daily slots (configurable):

```
08:50 – 09:00  Morning
11:50 – 12:00  Lunch
16:50 – 17:00  Evening
20:50 – 21:00  Prime time
22:50 – 23:00  Late night
```

Scheduling rules:

- **Birthday wishes:** Auto-scheduled to closest slot to 9 AM on EXACT birthday date
- **Marriage anniversaries:** Same
- **Upcoming marriage:** Scheduled 7d, 3d, and on-day
- **Events:** Scheduled 2d, 1d, and on-day
- **NotebookLM:** Routed to appropriate-level slot
- **General news:** Round-robin distribution

Slot held vs released (per Gnana):

- Slot HELD if AI processing/callback fails
- Show "Standby filler" instead of empty broadcast
- Super admin can force-release

Pre-broadcast notification:

10 min before each slot → push to all uploaders whose content airs.

14. Push Notification Templates

(Unchanged — see §14 of original plan)

15. AI Pipeline Webhook Integration (UPDATED v1.3)

v1.3 additions (per Sameer's clarification note + Gyan's `ai_callbacks_outbox`):

Environment URLs (Sameer Q1):

Env	Webhook (Admin → AI)	Callback (AI → Admin)
Staging	<code>https://pipeline-staging.localaitv.com/webhook/content-approved</code>	<code>https://api-staging.localaitv.com/api/webhook/ai-processed</code>
Production	<code>https://pipeline.localaitv.com/webhook/content-approved</code>	<code>https://api.localaitv.com/api/webhook/ai-processed</code>

Both env-configurable (`ADMIN_WEBHOOK_ALLOWED_ORIGIN` , `AI_CALLBACK_URL`). Phase 1 tests on staging first. All routes via Cloudflare.

Secret exchange (Sameer Q2/Q3):

- HMAC secret + callback bearer token generated once (`openssl rand -hex 32`)
- Admin side: stored in **Supabase Vault**
- AI side: stored in **AWS Secrets Manager** (EC2-native) — env vars `WEBHOOK_HMAC_SECRET` , `ADMIN_CALLBACK_TOKEN`
- Exchanged via **1Password shared vault** (never email/Slack/git)
- Rotation: dual-accept old+new for 24h grace; 48h advance notice. Runbook in §16.

Reliable callback delivery (Gyan §4.7): AI side uses the `ai_callbacks_outbox` transactional outbox table. Callback written to outbox in the same DB transaction as job completion → background sender delivers with retry → marks `delivered` or `dead_lettered` . Guarantees no lost callbacks even if Admin is briefly down.

15.1 Architecture (per Sameer + Gnana's contributions)



15.2 Webhook Payload Schema (Pydantic)

```
class MediaItem(BaseModel):
    type: Literal["photo", "video"]
    url: str # 1-hour signed URL
    size_bytes: int
    duration_seconds: Optional[int] = None
    thumb_url: Optional[str] = None
    sha256: str

class ApprovedContentWebhook(BaseModel):
    content_id: str # UUID v4
    approved_version: int = 1
    idempotency_key: str # = content_id + "_v" + approved_version
    job_id: Optional[str] = None

    geography_level: Literal["national", "state", "district", "constituency"]
    state: Optional[str] = None
    district: Optional[str] = None
    constituency: Optional[str] = None

    category: Literal["news", "birthday", "marriage_day", "upcoming_marriage",
                      "event", "shopping", "car_sale", "rental", "job"]
    headline: str = Field(min_length=4, max_length=300)
    description: str = Field(min_length=20, max_length=3000)
    location: Optional[str] = None
    form_data: dict = Field(default_factory=dict)

    media_urls: list[MediaItem] = Field(min_length=1, max_length=3)

    author_id: str
    author_role: Literal["citizen", "admin", "master_admin", "super_admin"]
    approved_by: str
    approved_at: datetime

    target_channel: str # e.g. "kurnool-tv" - selects TTS_PROVIDER_<CHANNEL>
    processing_mode: Literal["ai_pipeline", "raw_video", "notebooklm_generated"]
```

15.3 Webhook Response (AI → Admin)

```
{
  "status": "queued",
  "ok": true,
  "content_id": "abc-123-def",
  "approved_version": 1,
  "job_id": "celery-task-id",
  "idempotency_status": "new", // | "already_queued" | "already_completed" | "rejected"
  "accepted_at": "2026-05-14T10:30:00Z",
  "estimated_completion_at": "2026-05-14T10:32:00Z"
}
```

15.4 Callback Format (AI → Admin)

```
POST /api/webhook/ai-processed
Authorization: Bearer <shared_callback_token>
X-LocalAITV-Idempotency-Key: <same>

{
  "content_id": "abc-123-def",
  "approved_version": 1,
  "job_id": "celery-task-id",
  "status": "ready_for_bulletin", // or failed_retryable/failed_final
  "ai_output_url": "https://r2.../ai-processed/AP/Kurnool/Kurnool-Urban/news/abc-123/final.mp4",
  "thumbnail_url": "https://r2.../thumb.jpg",
  "duration_seconds": 47,
  "processed_at": "2026-05-14T10:35:00Z",
  "error_message": null,
  "metrics": {
    "gemini_time_ms": 1240,
    "tts_time_ms": 3820,
    "ffmpeg_time_ms": 12500,
    "total_time_ms": 17560
  }
}
```

15.5 Cancel Job Endpoint

```
DELETE /webhook/cancel-job/:job_id
```

Possible responses:

```
200 OK { "cancelled": true, "stage": "queued" | "processing" }
410 Gone { "error": "already_complete", "ai_output_url": "..." }
404 Not Found { "error": "job_not_found" }
```

15.6 HMAC Signing

```
# Sender (Admin Dashboard)
body_bytes = json.dumps(payload, sort_keys=True).encode('utf-8')
signature = hmac.new(SHARED_SECRET, body_bytes, hashlib.sha256).hexdigest()

# Receiver (AI Pipeline) – must verify using raw body bytes
raw_body = await request.body() # NOT request.json()
expected_sig = hmac.new(SHARED_SECRET, raw_body, hashlib.sha256).hexdigest()
if not hmac.compare_digest(expected_sig, request.headers['X-LocalAITV-Signature']):
    return JSONResponse({"error": "invalid_signature"}, status_code=401)
```

15.7 Retry Policy + Callback Failure Contract (Sameer Q4 — definitive)

Webhook delivery (Admin → AI):

Failure type	Action
HTTP 5xx response	Retry: 1s → 5s → 30s → 5min → 30min
Network timeout	Retry same schedule
HTTP 4xx (validation)	DO NOT retry — write to dead-letter, alert super admin
HTTP 200 OK	Mark delivered
HTTP 200 <code>idempotency_status: already_queued</code>	Mark delivered (duplicate is fine)

Callback delivery (AI → Admin) — the Sameer Q4 contract:

Callback response	AI Pipeline action	Admin Dashboard action
2xx success	Mark <code>ai_processing_jobs</code> complete; <code>ai_callbacks_outbox.status=delivered</code>	<code>content.status = 'ready_for_bulletin'</code>
5xx (Admin temporarily down)	RETRY — 5 attempts exp backoff (transient)	Recovers; watchdog also catches
4xx (bad payload from AI)	DO NOT RETRY — <code>ai_callbacks_outbox.status=dead_lettered</code> + Sentry. Preserve S3 output.	Content stays <code>ai_processing</code>
Timeout / no response	Treat as 5xx — retry	Watchdog catches after 30 min

Stuck-content reconciliation (the definitive answer to "does content stay in ai_processing forever?"):

No. Admin Dashboard `pg_cron` watchdog runs **every 5 minutes**:

1. Finds `content.status = 'ai_processing'` for **> 30 minutes** with no successful callback
2. Sets `content.status = 'callback_lost'`
3. Alerts super admin
4. Super admin reconciles:
 - **Video IS in S3** (`ai-processed/...`)? → "Confirm processed" → `ready_for_bulletin` (processing was fine; only callback wire failed)
 - **Video NOT in S3?** → "Re-queue" → `content.status = 'approved'` → new webhook with **fresh idempotency_key** (won't be rejected as duplicate)

AI Pipeline error-handling rule: On a 4xx callback, log + dead-letter, but **do NOT mark the content as failed on your side** — the video may be perfectly valid in S3. The Admin watchdog reconciles. Preserve the S3 output + the `ai_callbacks_outbox` row.

After 5 webhook attempts → dead-letter → Sentry alert + super admin notification.

16. Security & Row Level Security (unchanged from v1.0 — see §16)

16a. Security Hardening (Abishek's input)

Threat	Solution	Cost
Bot uploads / scraping	Cloudflare Turnstile on all forms	₹0
DDoS / Layer 7 attacks	Cloudflare WAF (auto with R2)	₹0
API rate limiting	Per-IP token bucket in Netlify Functions	₹0
OTP abuse	Per-phone (5/hr) + Per-IP (20/hr) on <code>/api/sms-otp-send</code>	₹0
Admin brute-force	Lockout after 5 failed logins; IP block after 20/hour	₹0
Session hijack	HttpOnly + Secure + SameSite cookies, 24h expiry	₹0
File upload abuse	Per-user daily quota (20 uploads/day default)	₹0
Compromised credentials	HIBP API on admin signup	₹0
XSS	React auto-escape + CSP headers	₹0
SQL injection	Supabase parameterized queries (built-in)	₹0
Anomaly detection (Phase 2)	PostHog event rules	₹0 (free tier)
Force-logout suspect admin	Super admin "kill session" action	₹0

17. Audit Log (unchanged — see §17 of v1.0)

17a. Queue Infrastructure

pg_boss (Postgres-based job queue)

```
-- Installed as Supabase extension
CREATE EXTENSION pgboss;
```

Job queues

Queue name	Purpose	Concurrency
webhook_delivery	Admin → AI Pipeline webhook calls	10 workers
ai_callback_handler	Processing AI callbacks	5 workers
notification_delivery	Push notifications via FCM	20 workers
media_validation	Virus scan, NSFW, hash dedup	10 workers
analytics_refresh	Materialized view refreshes	1 worker
cleanup	R2 lifecycle cleanup, deleted content	1 worker

Idempotency

Every job has an `idempotency_key`. `pg_boss` deduplicates automatically.

Retry policy

- 5 attempts with exponential backoff: 1s → 5s → 30s → 5min → 30min
- After exhaustion → moved to `dead_letter` queue → Sentry alert

Watchdog (pg_cron)

```
-- Runs every 5 minutes
SELECT cron.schedule('stuck-job-watchdog', '*/*5 * * * *', $$
  -- Re-queue items stuck in ai_processing > 30 min
  UPDATE content SET status = 'callback_lost'
  WHERE status = 'ai_processing' AND updated_at < NOW() - INTERVAL '30 minutes';

  -- Re-queue failed_retryable jobs older than 1 hour
  UPDATE webhook_deliveries SET attempt_count = 0
  WHERE last_response_status >= 500 AND last_attempted_at < NOW() - INTERVAL '1 hour';
$$);
```

At scale

When `pg_boss` insufficient (>10K jobs/min sustained):

- Migrate to **Cloudflare Queues** (\$0.40/million messages)
- Or self-hosted **Redis + BullMQ**

17b. Observability & Monitoring

Tool	Purpose	Free tier	Cost @ scale
Sentry	Error tracking, stack traces, releases	5K errors/mo	₹0–2K
Better Stack (Logtail)	Log aggregation	1 GB/mo	₹0–3K
PostHog	Product analytics, session replay, feature flags	1M events/mo	₹0–25K
Cloudflare Analytics	CDN + R2 metrics, request analytics	Built-in	₹0
Supabase Logs	DB queries, slow query log	Built-in	₹0
Netlify Function Logs	Serverless function logs	Built-in	₹0
OpenTelemetry	Distributed tracing (Phase 3)	Self-hosted	₹0
Grafana Cloud	Custom dashboards (queue depth, p95 latency)	10K series	₹0
Flower (Celery monitoring)	AI Pipeline queue + worker visibility	Self-hosted	₹0

Critical alerts (PagerDuty / Slack)

Alert	Threshold
Webhook delivery dead-letter	>0 in last 15 min
AI callback missing	Content in ai_processing >30 min
Queue depth	>500 pending in any queue
Worker count	Below configured minimum
5xx response rate	>1% in last 5 min
Admin login failures (per IP)	>20/hour
Sentry error rate	>50/hour

18. Tech Stack Summary (v1.2 — broadcast scale)

Layer	Service	Today (9 ch)	Month 1 (300 ch)	Month 6 (3000 ch)
AI Pipeline compute	AWS EC2 g4dn.xlarge (Mumbai, NVIDIA T4 GPU)	₹30K	₹3L	₹15L
Storage	AWS S3 Mumbai (with lifecycle)	₹5K	₹4L	₹50L
CDN	Cloudflare (free → Pro)	₹0	₹0	₹0–20K
Database + Auth	Supabase (Mumbai)	₹0	₹2,100	₹2,100
Admin Dashboard hosting	Hostinger VPS (existing)	included	included	included
Job queue	pg_boss (in Supabase)	₹0	₹0	₹0
WAF + CAPTCHA	Cloudflare (free)	₹0	₹0	₹0
Push notifications	Firebase Cloud Messaging	₹0	₹0	₹0
SMS OTP	2Factor.in	₹400	₹2,500	₹25K
AI script gen	Google Gemini Flash	₹2K	₹50K	₹5L
AI voice (TTS)	Google Cloud TTS	₹3K	₹1L	₹10L
NSFW detection	Google Cloud Vision	₹0	₹5K	₹50K
YouTube (live only)	YouTube Data API	₹0	₹0	₹0
Error tracking	Sentry	₹0	₹0	₹2K
Log aggregation	Better Stack	₹0	₹0	₹3K
Product analytics	PostHog	₹0	₹0	₹25K
Email (optional)	Resend	₹0	₹0	₹0
Domain	localaitv.com	₹85	₹85	₹85
TOTAL		~₹45K/mo	~₹14L/mo	~₹1.3 CR/mo

19. Cost Projections (v1.2 — broadcast scale)

Without Cloudflare CDN (DO NOT do this — for reference only)

Phase	Channels	Storage	S3 egress alone	Total
Today	9	₹5K	₹50K	₹50K
Month 1	300	₹4L	₹50L	₹59L
Month 6	3,000	₹50L	₹5-10 CR	₹5-10 CR ❌

With Cloudflare CDN (the v1.2 architecture)

Phase	Channels	Compute	Storage	CDN	Egress	AI APIs	Total
Today	9	₹30K	₹5K	₹0	₹500	₹5K	₹45K
Month 1	300	₹3L	₹4L	₹0 (free)	₹5L	₹1.5L	₹14L
Month 3	1,000	₹8L	₹15L	₹20K	₹15L	₹5L	₹45L
Month 6	3,000	₹15L	₹50L	₹50K	₹50L	₹15L	₹1.3 CR

Per-video lifetime cost at peak scale

Cost component	Per video
Generation (EC2 + Gemini + TTS)	₹3-5
Storage (1 year, with lifecycle)	₹0.50
Delivery (avg 1,000 views via CDN)	₹0.10
Total per video lifetime	₹4-6

At 600,000 videos/day at peak: **₹2.4 - 3.6 crore/month operational cost** (matches the table above).

Why this is sustainable

Cloudflare CDN saves ₹5-9 crore/month at peak by serving 90%+ of requests from cache. Without it, the architecture is economically infeasible. With it, broadcast scale becomes affordable.

Revenue requirement to be sustainable

Assuming 60% gross margin target:

- Month 1: Need ~₹35L/mo revenue
- Month 6: Need ~₹3.3 CR/mo revenue

Potential revenue streams to consider:

- Constituency-level advertising
- Government / political party content sponsorship
- Premium tier for content creators
- Citizen reporter compensation pool (one-time admin fees)
- Programmatic advertising on app feed

20. Implementation Phases (v1.3 — with Sameer's committed delivery plan)

Phase 0 — URGENT (this week) — Fix 15min → 90sec bottleneck

Goal: Get AI Pipeline off shared VPS onto dedicated GPU compute.

CONFIRMED: Pure lift-and-shift, ZERO pipeline logic changes. The only adjustment is one FFmpeg flag for GPU acceleration: `-c:v libx264` → `-c:v h264_nvenc -preset p4 -tune hq`. This single change is what drops bulletin generation from ~15 min → ~90 sec. Gemini calls, TTS calls, Celery structure, RabbitMQ queues, S3 I/O, build/maintenance workers — all run exactly as-is.

Sameer's committed Phase 0 plan (owners assigned by AI team):

Task	Owner
Launch EC2 g4dn.xlarge in <code>ap-south-1</code> , install CUDA + FFmpeg-NVENC + Python 3.11	Sameer
Migrate AI pipeline code from Hostinger VPS to EC2 (no code changes)	Gnana
Swap FFmpeg encoder flag to <code>h264_nvenc</code>	Gnana
Test end-to-end: bulletin generation target < 2 minutes	Sameer + Gnana
Verify S3 read/write from EC2 instance	Sameer

- Effort:** 2-3 days
- Cost:** ₹30K/mo for 24/7 instance
- Dependency:** None — can start immediately, does not depend on Admin Dashboard build

Phase 1 — Admin Dashboard + Webhook Adapter (Sameer's committed plan)

AI team's committed webhook-adapter tasks (Phase 1, owners assigned):

Task	Owner
POST /webhook/content-approved endpoint (FastAPI)	Sameer
HMAC signature verification (raw body)	Sameer
Idempotency check via content_id + approved_version	Sameer
S3 signed-URL media download adapter	Gnana
Output upload to ai-processed/ folder structure	Gnana
POST /api/webhook/ai-processed callback to Admin	Sameer
Callback retry: 5 attempts exp backoff + ai_callbacks_outbox	Sameer
Direct-publish lane (admin + NotebookLM bypass)	Gnana
OpenTelemetry instrumentation (all 5 stages)	Sameer
Separate Celery queue: tts_realtime + tts_high_volume	Gnana
Fallback policy: Gemini→OpenAI, TTS→Sarvam	Sameer
First-phase DB tables (Gyan's 10 — \$7b.9)	Gnana

Existing pipeline (Gemini, TTS, FFmpeg, build worker) requires zero changes. All Phase 1 AI-side work is purely the integration adapter + DB tracking layer.

Phase 1 — Admin Dashboard + 300-channel infrastructure

Phase 1 — Admin Dashboard + 300-channel infrastructure

Goal: Production-grade admin moderation + scale infrastructure for 300 channels.

Week 1:

- Supabase project (Mumbai)
- Database schema (all tables + indexes + RLS)
- pg_boss queue setup
- Sentry + Better Stack integration
- Cloudflare account setup (DNS + WAF + Turnstile + CDN)
- AWS S3 bucket structure + lifecycle policies
- Auth flow (password + SMS OTP)
- One super admin account (founder)

Week 2:

- Admin Dashboard skeleton (React) on Hostinger VPS
- FastAPI backend on Hostinger VPS
- Review queue with state-filtering
- Item detail / review screen
- Claim/lock mechanism (atomic SQL)
- Approve / Reject flows
- Webhook delivery via pg_boss queue
- Idempotency + dead-letter handling

Week 3:

- Modify & Approve (original retention)
- Escalation flow
- Admin direct upload
- Push notification integration (FCM)
- Notifications to citizens
- Media validation pipeline (MIME + virus + hash + NSFW)

Week 4:

- AWS EC2 Auto Scaling Group (3-5 instances based on queue depth)
- Cloudflare CDN configured in front of S3 (mandatory before going to 300)
- Broadcast slots config
- pg_cron + advisory locks scheduler
- Birthday auto-scheduling
- NotebookLM upload (5 sub-types)
- Publishing queue UI
- Onboard 5-10 initial admins for AP + TG

End of Phase 1: 300 channels operational. ~₹14L/mo cost.

Phase 2 — Scale to 1,000 channels (Month 1-3)

- Expand to additional states (Karnataka, Tamil Nadu, Kerala, etc.)
- Increase AWS EC2 ASG to 10-15 instances
- Upgrade to Cloudflare Pro (\$20/mo) for advanced rules + analytics
- Add S3 Transfer Acceleration for faster uploads
- Set up AWS CloudWatch + Grafana for operational monitoring
- Analytics dashboard with drill-down
- Reporter profile pages
- Team management UI
- Onboard regional master admins
- Load test (k6 / Artillery) at 1,000-channel volume
- Disaster recovery runbook

Phase 3 — Pan-India 3,000 channels (Month 3-6)

- AWS EC2 ASG scales to 50+ instances during peak hours
- Consider Cloudflare Stream for some transcoding workloads if FFmpeg bottlenecks
- Optional: Cloudflare R2 hybrid for "warm" content tier (30-90 days old)
- Regional content replication (multi-region S3 if needed)
- Real-time operational dashboards
- Onboard 1,000+ admins across all states
- Production-readiness checklist final pass
- Full pan-India launch

Phase 4 — Optimization (Month 6+)

- Reporter ranking auto-approval (90%+ approval rate)
- AI-assisted moderation
- Bulk approve / reject
- Comment moderation
- Reporter compensation system (when monetization model ready)
- Public analytics / leaderboards

20a. Production Readiness Checklist

Before declaring v1 launched:

- ☐ Load test (1K concurrent admins, 10K citizen uploads/hour)
 - ☐ Smoke tests for every endpoint
 - ☐ Disaster recovery runbook documented
 - ☐ Rollback procedure tested
 - ☐ All Sentry alerts firing correctly to Slack
 - ☐ Cloudflare WAF rules tuned to actual traffic
 - ☐ Rate limits validated against real usage
 - ☐ R2 lifecycle rules applied
 - ☐ Backup strategy documented (Supabase auto-backup confirmed)
 - ☐ Admin onboarding doc tested with non-technical user
 - ☐ AI Pipeline webhook integration tested end-to-end with all 9 categories
 - ☐ Cancel-job flow tested at each stage
 - ☐ Idempotency tested (duplicate webhook sends)
 - ☐ Failover behavior tested (kill worker mid-job)
-

21. Team Requirements

Required:

- **Founder (Koneti Mohan Reddy)** — decisions, API keys, approval
- **AI Team (Sameer + Gnana)** — webhook adapter, ~1 week effort, existing pipeline unchanged
- **Claude (Anthropic)** — admin dashboard + database + integrations
- **Form Engineer (Megan)** — align live app forms with documented schema

NOT required:

-  DevOps engineer
-  Full-stack hire (Claude builds)
-  DBA (Supabase managed)

Optional later:

- Part-time React maintainer (5-10 hrs/week post-launch)
-

22. What's IN / OUT of v1

IN v1 (additions from v1.0)

- All v1.0 features
- **Production-grade queue infrastructure** (pg_boss)
- **Observability stack** (Sentry + Better Stack + PostHog)
- **Security hardening** (Turnstile, WAF, rate limits, account lockout, file quotas)
- **Media validation** (MIME, virus, hash, NSFW)
- **Idempotency + HMAC + retry** in webhook contract
- **Bulletin output structure** per Gnana's spec
- **Retention policy** (R2 lifecycle rules)

- **Cancel-job flow** for super admin
- **Production readiness checklist**

✗ OUT of v1 (deferred)

- Public analytics / leaderboards
- AI-assisted moderation (auto-flag)
- Reporter compensation/payout
- Reporter ranking-based auto-approval (Phase 2 — data tracked now)
- Bulk approve/reject
- Comment moderation
- Multi-language admin UI chrome
- Realtime websocket queue (polling sufficient for v1)
- ClickHouse OLAP (Phase 2 when needed)

23. What I Need From You — Action Items (v1.2 final list)

Phase 0 (this week — IMMEDIATE, before everything else)

#	Item	Time	Cost	Owner
0.1	AWS root account access OR existing AWS account credentials	Confirm	(existing)	You
0.2	Launch AWS EC2 g4dn.xlarge in ap-south-1 (Mumbai)	30 min	~₹30K/mo	AI team
0.3	Verify AI pipeline code can run on EC2 (test bulletin generation)	1 day	included	AI team

Phase 1 (Week 1-4)

#	Item	Time	Cost
A.	Supabase project (Mumbai) — https://supabase.com/dashboard	5 min	₹0
B.	Cloudflare account (sign up at https://dash.cloudflare.com)	10 min	₹0
B.1.	Add localaitv.com to Cloudflare DNS (point to current Hostinger IP)	10 min	₹0
B.2.	Add content.localaitv.com → CNAME to S3 bucket (will route via CDN)	5 min	₹0
B.3.	Enable Cloudflare Turnstile (CAPTCHA) — generate site key	5 min	₹0
C.	Firebase Cloud Messaging project — https://console.firebase.google.com	10 min	₹0
D.	Confirm Google Gemini Flash API access	2 min	(existing)
E.	Confirm Google Cloud TTS API access	2 min	(existing)
F.	2Factor.in (you already have this)	0	(existing)
G.	Hostinger VPS — confirm SSH access for admin dashboard deploy	5 min	(existing)
H.	DNS: admin.localaitv.com → CNAME to Hostinger VPS IP (via Cloudflare)	5 min	₹0
I.	AI Pipeline webhook URL (e.g. https://pipeline.localaitv.com/webhook/content-approved)	(their config)	(existing infra)
 K.	Sentry account — https://sentry.io	5 min	₹0 (free 5K errors/mo)
 L.	Better Stack (Logtail) account — https://betterstack.com	5 min	₹0 (free 1 GB/mo)
 M.	PostHog account — https://posthog.com	5 min	₹0 (free 1M events/mo)
 N.	YouTube Data API project + OAuth — https://console.cloud.google.com	30 min	₹0
 O.	9 YouTube channel credentials (one per district TV)	1 hour total	₹0

Total founder time

- **Phase 0 (urgent):** ~30 min (provide AWS credentials, approve EC2 launch)
- **Phase 1:** ~1.5 hours across all signups + DNS changes

Costs

- **Phase 0 immediate:** ~₹30K/mo (EC2 instance)
- **Phase 1 launch (300 channels):** ~₹14L/mo total
- **All free-tier services** cover their respective workloads for first 6 months

What I do NOT need from you

-  Code (Claude writes everything)
-  DevOps engineer (managed services handle ops)
-  Full-stack developer (Claude + AI team handle full stack)
-  GCP setup (we use Google APIs but not GCP infrastructure)

24. Approval & Sign-off

Founder approval

Item	Confirmed by founder?
Three-tier admin + state restrictions	☐
Form fields (Megan-aligned during Phase 1)	☐
Supabase + R2 + pg_boss + Sentry stack	☐
Webhook contract (HMAC + idempotency + retries)	☐
Bulletin output structure (Gnana's spec)	☐
Retention policy (48hr bulletins, 30d audit)	☐
Security hardening (Turnstile, WAF, rate limits)	☐
Push notifications (FCM, free)	☐
NotebookLM 5 sub-types	☐
Analytics dashboard with drill-down	☐
Implementation phased build	☐
₹2,000–5,500/mo infrastructure cost	☐
OUT of v1: payout, leaderboards, AI moderation, realtime	☐

AI Team approval

Item	Confirmed?
Webhook contract (FastAPI intake, 202 fast, async Celery)	☐
Idempotency = content_id + approved_version	☐
Cancel-job endpoint feasibility	☐
Callback retry policy (5 attempts)	☐
Bulletin output structure adoption	☐
R2 download adapter implementation (~1 day)	☐
Direct-publish lane (admin/notebooklm)	☐
OpenTelemetry instrumentation	☐
Fallback policies (Gemini → OpenAI, GCP TTS → Sarvam)	☐
Separate Celery queue for high-volume TTS	☐

Final approval to begin Phase 1:

Founder signature: _____ Date: _____

Contact

Founder: Koneti Mohan Reddy **Email:** localaitv@gmail.com **Company:** LocalAI Media Network Pvt Ltd **CIN:** U63910KA2025PTC212593

Plan version: v1.1 final **Generated:** 14 May 2026 **Supersedes:** v1.0 **Subject to:** mutual iteration before Phase 1 build begins